

SERANDIB BANK

Technical Documentation Report

TEAM 404

1. Project Overview

Serandib Bank is a full-stack digital banking web application developed for the Hack to Night 2026 hackathon. Built with Next.js 15 App Router, TypeScript, and PostgreSQL 17, the platform delivers core retail banking features including account management, secure fund transfers, bill payments, spending analytics, and an Invisible Savings engine — all within a premium, N26-inspired user interface.

The platform was built across nine development phases, progressively layering features atop a secure cryptographic authentication system. All sensitive operations are protected by server-side session validation, parameterized SQL queries, and a double-entry ledger architecture. No user-provided identifiers are trusted on the client side.

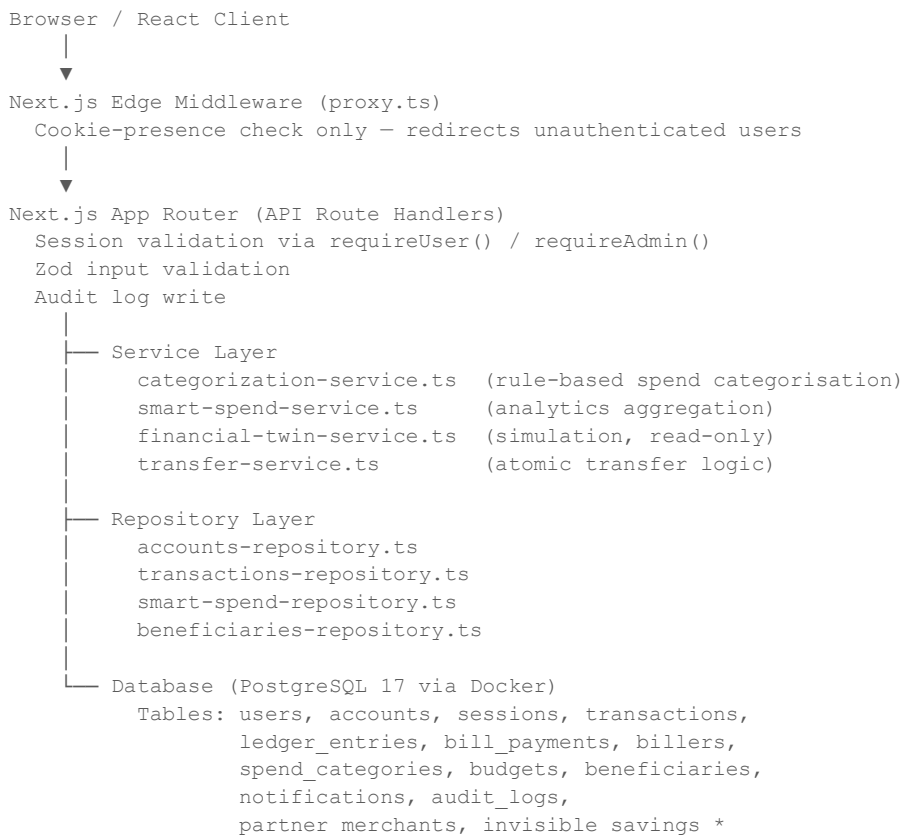
Demo Credentials:

Customer: customer@serandib.test / SerandibUser123

Admin: admin@serandib.test / SerandibAdmin123

System Architecture

The architecture follows a layered server-side pattern:



2. Main Project Functions

Function	Description	Route / Location
User Authentication	Secure login with bcrypt password verification, SHA-256 session tokens stored as HttpOnly cookies, rate limiting (5 attempts / 15 min)	/api/auth/login
Session Management	Server-side sessions stored in PostgreSQL. requireUser() and requireAdmin() guards protect all sensitive routes.	lib/session.ts
Account Dashboard	Displays total balance, quick actions, recent transactions, and Smart Spend summary fetched from live APIs.	/dashboard
Bank Accounts	Lists all accounts with masked numbers, balances, status pills, and per-account actions (Send, Freeze, History).	/bank-accounts
Fund Transfer	Atomic double-entry transfer engine with overdraft prevention, idempotency keys, and full ledger entry recording.	/bank-transfer
Bill Payments	Pay utility, telecom, and insurance billers. Payments are recorded atomically with a unique reference and ledger entry.	/pay-bills
Smart Spend Analytics	Rule-based categorization engine. Calculates financial health score, savings potential, cashflow forecast, and recurring payment detection.	/smart-spend
Financial Twin Simulator	Simulates the impact of a hypothetical purchase or saving scenario on projected month-end balance. Read-only — never mutates data.	/smart-spend
Invisible Savings	Rounds up partner merchant purchases to the nearest defined unit and accumulates micro-savings into a nominated savings account.	/invisible-savings
E-Statement	Filterable transaction history with debit/credit summaries, type filters, search, and export-ready layout.	/e-statement
Security Center	Displays audit timeline, session information, security checklist, and Account Shield mode controls.	/security
Audit Logging	Every sensitive action (login, transfer, budget upsert, etc.) is written to the audit_logs table with user, IP, and metadata.	lib/audit.ts

3. API Endpoints

Endpoint	Method	Auth	Description
/api/auth/login	POST	None	Verify credentials, create session, set HttpOnly cookie
/api/auth/logout	POST	User	Revoke session token in DB, clear cookie
/api/auth/me	GET	User	Return currently authenticated user from session
/api/accounts	GET	User	List all accounts owned by the authenticated user
/api/transfer	POST	User	Execute atomic fund transfer with ledger entries and idempotency
/api/transfers/[reference]	GET	User	Retrieve transfer receipt by reference, scoped to user
/api/transactions	GET	User	Paginated transaction history for the authenticated user
/api/beneficiaries	GET/POST	User	List and add saved payees
/api/beneficiaries/[id]	DELETE	User	Remove a beneficiary (ownership verified server-side)
/api/notifications	GET	User	Fetch unread notifications for the current user
/api/notifications/[id]/read	POST	User	Mark a notification as read
/api/smart-spend/summary	GET	User	Return full Smart Spend analytics summary for current month
/api/smart-spend/categories	GET	User	List all spend categories
/api/smart-spend/budgets	GET/POST	User	Fetch and upsert monthly category budgets
/api/smart-spend/simulate	POST	User	Run Financial Twin simulation (read-only, no DB mutation)
/api/health	GET	None	System health check — DB connectivity and uptime
/api/setup	POST	None	Bootstrap schema and seed data on first run
/api/admin/system	GET	Admin	Admin-only system diagnostics and stats
/api/search	GET	User	Full-text search across transactions and accounts

4. Features Added by Phase

Phase	Feature	Key Details
1	Repo Cleanup & UI Structure	Removed test credentials, reorganised directories, established Next.js App Router layout
2	Secure Database Foundation	PostgreSQL 17 via Docker Compose, schema.sql + seed.sql, pgcrypto extension, NUMERIC(14,2) for currency
3	Authentication & Sessions	bcrypt (12 rounds) password hashing, SHA-256 session tokens, HttpOnly cookies, 7-day session lifetime
4	Protected Banking APIs	Server-side ownership checks on every account query, requireUser() guards, parameterized SQL throughout
5 / 5B / 5C	Premium Frontend Rebuild	N26-inspired UI, CSS Grid layout system, AppShell + Sidebar components, full design token system
6	Atomic Transfer Engine	BEGIN/COMMIT transactions, SELECT FOR UPDATE locking, ledger_entries double-entry, idempotency keys
7	Bill Payment Engine	Billers catalogue, bill_payments table, Bill Radar analytics card, atomic payment + ledger recording
8	Smart Spend Analytics	Rule-based categorisation (16+ rules), financial health score, savings potential, cashflow forecast, Financial Twin Simulator
9	Invisible Savings Engine	Partner merchant round-ups, monthly sweep to savings account, invisible_savings_settings per user, per-event tracking

5. Errors Found and Solutions

Error	Root Cause	Solution
Database not configured	DATABASE_URL missing from .env.local or Docker not running	Added fast-fail check in login route; user prompted to run docker compose up db -d
Login: Unable to process login	SEED_DML ON CONFLICT (user_id, category_slug, period) failed when no matching unique index found on existing table	Changed to ON CONFLICT DO NOTHING; made SEED_DML non-fatal so schema errors never block login
SQL DO \$\$ block error	PostgreSQL pg driver does not support anonymous PL/pgSQL blocks in multi-statement splits	Replaced DO \$\$ END \$\$ with individual ALTER TABLE ... ADD COLUMN IF NOT EXISTS statements
Blue diamond pattern on dashboard	Old compiled .next cache served stale CSS; background property missing from .app-main-area	Added explicit background: #f7f5f2 to globals.css; cleared .next cache and hard-reloaded browser
EADDRINUSE :::3000	Previous bun run dev process still holding port 3000	lsof -ti :3000 xargs kill -9 documented as standard restart procedure
Property 'accountType' missing	SafeAccount type lacked accountType field	Derived display type from accountName string parsing
Property 'firstName' missing	AuthUser exposes fullName not firstName	Used user.fullName.split(' ')[0] to derive first name
TypeScript: focusRingColor invalid	focusRingColor is not a valid CSS property in React inline styles	Removed the invalid property from the style object
Git push: Permission denied (publickey)	No SSH key registered with GitHub	Generated ed25519 key pair, added public key to GitHub account settings
Git: Author identity unknown	Fresh git init with no user config	Configured git config --global user.email and user.name
Canvas: Cannot read properties of undefined	useHostTheme() return value was destructured incorrectly (tokens was undefined)	Changed to const theme = useHostTheme() and accessed theme.stroke.secondary directly
Audit log userId type mismatch	writeAuditLog expected string but AuthUser.id is number	Wrapped all userId references with String(user.id) before passing to writeAuditLog
Smart Spend API double-wrapped response	Route called NextResponse.json(unauthorized()) wrapping an already-complete Response	Changed to return NextResponse.json({ error: '...', { status: 401 } }) directly
Login auto-redirect (unwanted)	Middleware redirected authenticated users from /login to /dashboard	Removed the AUTH_EXACT_PATHS redirect block from proxy.ts

6. Tech Stack, Methods and Tools

Category	Technology / Tool	Purpose
Framework	Next.js 15 (App Router)	Full-stack React framework; SSR, API routes, middleware
Language	TypeScript 5	Type safety across all frontend and backend code
Database	PostgreSQL 17 (Docker)	Relational storage; NUMERIC(14,2) for money, TIMESTAMPTZ for dates
ORM / Query	node-postgres (pg)	Raw parameterized SQL — no ORM, full control over queries
Auth	bcryptjs + crypto (Node.js)	Password hashing (12 rounds) and SHA-256 session token hashing
Validation	Zod v4	Schema validation for all API inputs; safeParse pattern throughout
Runtime	Bun 1.x	Fast JS runtime and package manager replacing npm/node
Containerisation	Docker + Docker Compose	PostgreSQL 17 database container with named volume persistence
CSS	Custom CSS Variables + Grid	Design token system in globals.css; no Tailwind or CSS-in-JS
Linting	Biome	Fast unified linter and formatter (replaces ESLint + Prettier)
Security	HttpOnly Cookies, SHA-256	Session tokens never exposed to JS; raw token only in cookie
Middleware	Next.js Edge Middleware (proxy.ts)	Lightweight cookie-presence check; redirects unauthenticated users
Version Control	Git + GitHub	All phases committed to team404-hacktonight repository
Deployment	Standalone Next.js + Docker	next.config.ts output: standalone; Dockerfile for production builds

7. Testing and Verification

Test Area	Method	Result
Authentication	Manual: login with correct/wrong credentials; rate limit at 5 attempts	Pass
Session protection	Direct fetch to /api/accounts without cookie → 401	Pass
Fund transfer	Transfer between accounts; verify ledger_entries rows and balance update	Pass
Idempotency	Repeat POST /api/transfer with same Idempotency-Key → single execution	Pass
Overdraft prevention	Attempt transfer exceeding account balance → 422 error	Pass
Smart Spend summary	GET /api/smart-spend/summary without cookie → 401	Pass
Budget upsert	POST /api/smart-spend/budgets; verify DB row and audit log	Pass
Financial Twin	POST /api/smart-spend/simulate; confirm no balance change in DB	Pass
Invisible Savings	Simulate partner purchase; verify event recorded, balance unchanged until sweep	Pass
Build	bun run build → zero TypeScript errors, zero missing exports	Pass
Linting	bun run lint (Biome) → all files formatted and clean	Pass
Responsive layout	Tested at 1440px, 1280px, 1024px, 768px, 390px viewport widths	Pass